

JOBSHEET 5

Nama: Muhammad Zainur Roziqin

Absen/Kelas : 23/TI 1G

PRAKTIKUM 5.1:

Kode di Faktorial.java:

```
public class Faktorial {  
    int FaktorialBF(int n){  
        int fakto = 1;  
        for (int i = 1; i<=n; i++){  
            fakto = fakto * i;  
        }  
        return fakto;  
    }  
    int FaktorialDC(int n){  
        if(n==1){  
            return 1;  
        }else{  
            int fakto = n * FaktorialDC(n-1);  
            return fakto;  
        }  
    }  
    public Faktorial() {  
    }  
}
```

kode di FaktorialMain.java:

```
import java.util.Scanner;  
public class FaktorialMain {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Masukkan n: ");  
        int input = sc.nextInt();  
  
        Faktorial fk = new Faktorial();  
        System.out.println("Nilai Faktorial: " + input + "Menggunakan  
BF: " + fk.FaktorialBF(input));  
        System.out.println("Nilai Faktorial: " + input + "Menggunakan  
DC: " + fk.FaktorialDC(input));  
        sc.close();  
    }  
}
```

```
}
```

Hasil:

```
Masukkan n: 5
Nilai Faktorial: 5Menggunakan BF: 120
Nilai Faktorial: 5Menggunakan DC: 120
Nilai Faktorial: 5Menggunakan while: 120
```

Pertanyaan

1. Kode di if itu dia langsung return 1 jika nilai $n = 1$, nantinya nilai 1 ini akan dimasukkan ke else.
2. Mungkin, ini contohnya:

```
int FaktorialBFWhile(int n){
    int i = 1;
    int hasil = 1;
    while(i <= n){
        hasil = hasil * i;
        i++;
    }
    return hasil;
}
```

hasilnya:

```
Masukkan n: 5
Nilai Faktorial: 5Menggunakan BF: 120
Nilai Faktorial: 5Menggunakan DC: 120
Yang pake while: 120
```

3. Faktori *= i itu mengalikan dari i sampai n, lalu ditambah semua hasilnya. sedangkan faktori = n * faktorialDC(n-1) dia itu fungsi rekursif memanggil dirinya sendiri.
4. Kesimpulan BF menggunakan loop sedangkan DC menggunakan fungsi rekursif.

PRAKTIKUM 5.3

Kode di class pangkat:

```
public class Pangkat23 {
    int nilai, pangkat;
    public Pangkat23(int n, int p){
        nilai = n;
        pangkat = p;
    }
    int pangkatBF(int n, int p){
        int hasil = 1;
        for(int i = 0; i < n+1; i++){
            hasil = hasil * n;
        }
    }
}
```

```

        return hasil;
    }

    int pangkatDC(int a, int n){
        if(n == 1){
            return a;
        }else{
            if(n%2 == 1){
                return (pangkatDC(a, n/2) * pangkatDC(a, n/2) * a);
            }else{
                return (pangkatDC(a, n/2) * pangkatDC(a, n/2));
            }
        }
    }
}

```

Kode di pangkatMain:

```

import java.util.Scanner;
public class PangkatMain23 {
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.println("Masukkan jumlah elemen: ");
        int elemen = sc.nextInt();

        Pangkat23[] png = new Pangkat23[elemen];
        for(int i = 0; i < elemen; i++){
            System.out.println("Masukan nilai basis elemen ke-" +
(i+1)+" : ");
            int basis = sc.nextInt();
            System.out.println("Masukkan nilai pangkat ke-"+(i+1)+" :
");
            int pangkat = sc.nextInt();
            png[i] = new Pangkat23(basis, pangkat);
        }

        System.out.println("Hasil pangkat bruteforce: ");
        for(Pangkat23 p : png){
            System.out.println(p.nilai + "^"+p.pangkat+" : "+
p.pangkatBF(p.nilai, p.pangkat));
        }
    }
}

```

```

    }

    System.out.println("Hasil pangkat DC: ");
    for(Pangkat23 p : png){
        System.out.println(p.nilai + "^"+p.pangkat+": "+
p.pangkatDC(p.nilai, p.pangkat));
    }
}
}
}

```

Hasil:

```

Hasil pangkat bruteforce:
2^3: 8
4^5: 1024
6^7: 279936
Hasil pangkat DC:
2^3: 8
4^5: 1024
6^7: 279936

```

Pertanyaan:

1. Perbedaan pangkatBF() vs pangkatDC():
 - pangkatBF(int n, int p) seharusnya menghitung n^p dengan perkalian berulang (iteratif/brute force, $O(p)$).
 - pangkatDC(int a, int n) pakai rekursi divide-and-conquer: memecah pangkat jadi $n/2$ lalu menggabungkan lagi ($O(\log n)$)
2. Ada, di bagian “menggabungkan” hasil rekursi dengan perkalian:
 - a. Ganjil: $\text{pangkatDC}(a, n/2) * \text{pangkatDC}(a, n/2) * a$
 - b. Genap: $\text{pangkatDC}(a, n/2) * \text{pangkatDC}(a, n/2)$
3. Method dengan parameter (n, p) tetap relevan jika tujuan desainnya adalah membuat method dapat menghitung pangkat untuk input apa pun tanpa bergantung pada atribut objek.

contoh:

```

int pangkatBF() {
    int hasil = 1;
    for (int i = 0; i < pangkat; i++) hasil *= nilai;
    return hasil;
}

```

4. Uraian cara kerja pangkatBF() dan pangkatDC()
 - a. pangkatBF():
 - i. Inisialisasi hasil = 1.
 - ii. Melakukan perkalian berulang hasil *= nilai sebanyak pangkat kali.
 - iii. Mengembalikan hasil sebagai nilai $\text{nilai}^{\text{pangkat}}$.
 - b. pangkatDC():
 - i. Base case: jika $n == 1$, mengembalikan a.

- ii. Divide: memecah masalah menjadi pangkat $n/2$.
- iii. Combine: mengalikan hasil sub-masalah (kuadrat). Jika n ganjil, dikali a sekali lagi.

PRAKIKUM 5.4:

Kode di class Sum:

```
public class Sum23 {  
    public double keuntungan[];  
  
    public Sum23(int el){  
        keuntungan = new double[el];  
    }  
  
    double totalBF(){  
        double total = 0;  
        for(int i = 0; i < keuntungan.length; i++){  
            total = total + keuntungan[i];  
        }  
        return total;  
    }  
  
    double totalDC(double arr[], int l, int r){  
        if(l==r){  
            return arr[l];  
        }  
  
        int mid = (l+r)/2;  
        double lsum = totalDC(arr, l, mid);  
        double rsum = totalDC(arr, mid + 1, r);  
  
        return lsum + rsum;  
    }  
}
```

Kode di SumMain:

```
import java.util.Scanner;  
public class SumMain23 {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        System.out.println("Masukkan jumlah elemen: ");  
        int elemen = sc.nextInt();  
    }  
}
```

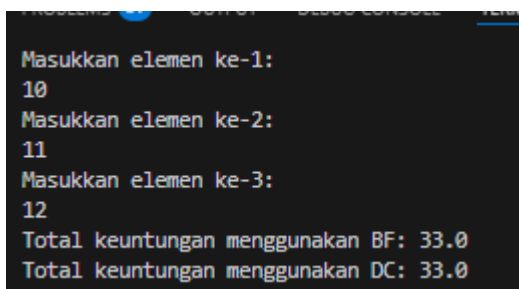
```

Sum23 sm = new Sum23(elemen);
for(int i= 0; i < elemen; i++){
    System.out.println("Masukkan elemen ke-"+(i + 1)+" : ");
    sm.keuntungan[i] = sc.nextDouble();
}

System.out.println("Total keuntungan menggunakan BF: "+
sm.totalBF());
System.out.println("Total keuntungan menggunakan DC: "+
sm.totalDC(sm.keuntungan, 0, elemen-1));
sc.close();
}
}

```

Hasil:



```

Masukkan elemen ke-1:
10
Masukkan elemen ke-2:
11
Masukkan elemen ke-3:
12
Total keuntungan menggunakan BF: 33.0
Total keuntungan menggunakan DC: 33.0

```

Pertanyaan:

1. mid dibutuhkan untuk membagi rentang indeks [l..r] menjadi dua sub-rentang (kiri dan kanan) pada metode totalDC(), yaitu [l..mid] dan [mid+1..r].
2. dipakai untuk rekursi menghitung total bagian kiri (lsum) dan total bagian kanan (rsum) dari array pada rentang yang sudah dibagi oleh mid.
3. Karena total keseluruhan pada rentang [l..r] adalah penjumlahan total sub-rentang kiri dan kanan, maka hasilnya harus digabung:
4. Base case totalDC() adalah saat rentang hanya berisi 1 elemen:
if (l == r) return arr[l];
Artinya kalau indeks kiri sama dengan indeks kanan, langsung kembalikan nilai elemen itu.
5. totalDC(arr, l, r) menjumlahkan elemen array pada rentang indeks [l..r] dengan strategi divide and conquer: bagi rentang jadi dua menggunakan mid, hitung total masing-masing bagian secara rekursif, lalu gabungkan dengan lsum + rsum sampai mencapai base case l == r.